

SISTEMA INTELIGENTE DE MONITORAMENTO

Missão Espacial Experimental — Relatório Técnico

Global Solution 2026 | FIAP · Fases 1, 2 e 3

Campo	Informação
Equipe	Eduardo Titato
Integrantes	Eduardo Titato — RM 572776
GitHub	github.com/edutitato10-rgb/GLOBAL-SOLUTION
Vídeo	https://youtu.be/E6gNNOrQBlw
Data	09 de junho de 2026
Versão	1.0

1. Introdução e Contexto

Este relatório documenta o desenvolvimento de um sistema inteligente de monitoramento para uma missão espacial experimental. O cenário simulado representa uma missão em órbita baixa com módulos críticos sofrendo degradação simultânea de energia, comunicação e radiação ao longo de um ciclo operacional de 10 horas.

O sistema recebe dados de telemetria em formato CSV, organiza as informações em estruturas de dados adequadas, aplica regras lógicas booleanas para diagnóstico, gera alertas automáticos priorizados e executa previsão por regressão linear para estimar o esgotamento energético — tudo implementado em Python puro, sem bibliotecas externas de ciência de dados.

2. Análise e Interpretação dos Dados

2.1 Pacote de Telemetria Simulado

O arquivo `data/dados.csv` contém quatro categorias de dados com estrutura uniforme (tipo, campo, valor, unidade, timestamp):

Categoria	Elementos	Qtd. Registros
Módulos críticos	suporte_vida, energia, comunicacao, habitat, laboratorio, armazenamento	6
Energia	geracao, consumo, reserva (6 horários cada)	18
Ambiente	temperatura_interna/externa, radiacao, qualidade_comunicacao	12
Log de eventos	8 eventos cronológicos	8

2.2 Inconsistência Proposital

O registro **SENSOR_FALHA** às 05:00:00 contém a leitura de temperatura externa de **-120°C**, fisicamente implausível para a superfície de Marte (faixa esperada: -130°C a +20°C). O sistema detecta essa anomalia automaticamente na seção de verificação de inconsistências e emite alerta de sensor com defeito, sem deixar que o valor contamine as análises de temperatura.

3. Estruturas de Dados

A escolha das estruturas reflete diretamente as características de cada conjunto de dados:

Estrutura	Uso	Justificativa
Dicionário	Status dos módulos; dados de energia e ambiente	Acesso O(1) por nome — essencial para diagnóstico rápido
Lista	Séries temporais de geração, consumo, reserva, temperatura	Sequência ordenada; suporta iteração para regressão linear

Fila (deque)	Alertas pendentes	FIFO — primeiro alerta gerado é o primeiro tratado
Pilha (list)	Últimos eventos críticos	LIFO — evento mais recente no topo para auditoria
Matriz	Energia x Horário	Representação bidimensional: linhas=horários, colunas=variaveis
Dict aninhado	Hierarquia da missão	Modela relação Missão → Subsistema → Componente

A hierarquia da missão (árvore implementada como dicionário aninhado) organiza os subsistemas em três grupos: **ENERGIA** (solar e baterias), **HABITAT** (oxigênio, temperatura, comunicação) e **OPERACOES** (laboratório e armazenamento). Essa estrutura permite navegação semântica e facilita a aplicação de regras por subsistema.

4. Lógica e Regras de Diagnóstico

4.1 Expressão Booleana Principal

A classificação do estado geral da missão em NORMAL, ALERTA ou CRÍTICO segue a expressão abaixo:

```
CRITICO = vida_em_risco OR energia_critica OR (radiacao_perigosa AND comunicacao_comprometida) ALERTA = (NOT CRITICO) AND (energia_em_alerta OR comunicacao_comprometida OR ...) NORMAL = NOT CRITICO AND NOT ALERTA
```

4.2 Detalhamento das Regras

Regra	Condição Booleana	Operadores
Energia crítica	reserva < 25% AND consumo > geracao	AND
Comunicação comprometida	NOT modulo_comunicacao OR sinal < 20%	NOT, OR
Radiação perigosa	radiacao > 100 mSv	-
Vida em risco	NOT suporte_vida OR NOT habitat	NOT, OR
Temperatura anormal	temp > 30°C OR temp < 15°C	OR
Estado crítico	vida_em_risco OR energia_critica OR (radiacao_perigosa AND comunicacao)	OR, AND

Cada regra é avaliada sequencialmente. O diagnóstico CRITICO tem precedência absoluta: uma vez que qualquer condição crítica é verdadeira, o sistema não reavalia as condições de alerta. Isso garante que situações de risco de vida nunca sejam downgraded por condições favoráveis em outros subsistemas.

5. Análise e Previsão de Dados

5.1 Método: Regressão Linear por Mínimos Quadrados

A técnica escolhida é a regressão linear simples, implementada do zero em Python puro sem bibliotecas externas. Dada a série temporal de reserva energética, os coeficientes a (inclinação) e b (intercepto) são calculados pelas fórmulas dos mínimos quadrados:

```
a = (n * sum(x*y) - sum(x) * sum(y)) / (n * sum(x^2) - sum(x)^2) b = (sum(y) - a * sum(x)) / n y_previsto = a * x + b
```

5.2 Resultados

Horário	Reserva Real (%)	Reserva Prevista (%)
T+0h (65.0%)	65.0	66.4
T+2h (58.0%)	58.0	57.9
T+4h (50.0%)	50.0	49.3
T+6h (43.0%)	43.0	40.7
T+8h (32.0%)	32.0	32.1

T+10h (22.0%)	22.0	23.6
T+12h — FUTURO	—	15.0 ■ CRITICO
T+14h — FUTURO	—	6.4 ■ CRITICO
T+16h — FUTURO	—	0.0 ■ CRITICO

A inclinação de **-8,571%/ciclo** confirma uma tendência de queda acentuada. A reserva atingirá zero em menos de 6 ciclos (12 horas), o que influencia diretamente a recomendação de desligar laboratório e sistemas não essenciais imediatamente.

6. Alertas Automáticos e Recomendações

O estado atual da missão é **CRÍTICO** — três condições críticas simultâneas foram detectadas:

#	Nível	Situação	Recomendação
1	CRITICO	Reserva em 22% com consumo > geracao	Desligar sistemas nao essenciais imediatamente
2	CRITICO	Comunicacao comprometida (sinal em 12%)	Ativar radio de emergencia e aguardar janela de contato
3	CRITICO	Radiacao em 110 mSv acima do limiar seguro	Recolher tripulacao para area blindada; suspender EVAs

Os alertas são ordenados por severidade (CRÍTICO > ALERTA > NORMAL) e exibidos com código de cores. A fila FIFO garante que alertas gerados anteriormente sejam processados antes dos novos; a pilha LIFO mantém os três eventos críticos mais recentes disponíveis para auditoria rápida.

7. Decisões Técnicas e Justificativas

- **CSV como fonte de dados:** Permite separação clara entre dados e lógica, facilita troca de cenários sem alterar o código. Suportado nativamente pelo módulo csv do Python.
- **Regressão linear sem libs externas:** Demonstra compreensão do algoritmo. O cálculo de mínimos quadrados com $n < 10$ pontos não requer NumPy e é mais didaticamente transparente.
- **Dicionário para módulos:** A chave é o nome do módulo; o valor é um bool (0/1). Acesso $O(1)$ é essencial quando o sistema precisa checar dezenas de módulos em tempo real.
- **deque para fila de alertas:** collections.deque oferece $O(1)$ para appendleft/pop, mais eficiente que list para operações FIFO em alta frequência.
- **Inconsistência detectada automaticamente:** A função detectar_inconsistencias() verifica faixas plausíveis para cada sensor antes que os valores entrem no diagnóstico principal, evitando que dados corrompidos gerem alertas falsos.

8. Conclusão

O sistema demonstra como conceitos fundamentais de computação — estruturas de dados, lógica booleana e algoritmos básicos de análise — podem ser combinados para resolver um problema realista de engenharia aeroespacial. A análise do cenário atual revela uma missão em situação crítica, com energia em esgotamento acelerado, comunicação comprometida e radiação acima dos limites seguros. As recomendações geradas automaticamente são coerentes com protocolos reais de missão espacial.

O projeto reforça a importância de validar dados antes de tomar decisões automatizadas (a inconsistência de -120°C é um exemplo concreto disso) e de comunicar resultados técnicos de forma clara e priorizada.